Speculative Decoding for Efficient LLM Inference

Samsu Sempena
The University of Sydney
ssem0956@uni.sydney.edu.au

Zhuohan Cui
The University of Sydney
zcui0321@uni.sydney.edu.au

Abstract

Speculative decoding improves large language model (LLM) inference throughput by drafting multiple candidate tokens with a smaller model and verifying them with a larger target model. However, reported gains vary substantially across hardware, model pairing, and decoding regime, and consumer-GPU evidence remains limited. We present a controlled empirical study on a single RTX 5090 Laptop using a Qwen2.5-3B-Instruct target and same-family 0.5B/1.5B drafts over GSM8K, a 5-subject MMLU subset, and CNN/DailyMail, with frozen manifests and matched prompts across all runs. We evaluate 12 speculative configurations ($k \in \{4, 8, 16\}$, deterministic/stochastic regimes, two draft sizes) plus regime-matched baselines, and report wall-clock latency, throughput, acceptance dynamics (α, B_{eff}), and task-level quality deltas. On the current artifact set, the highest mean speedup is observed for 1.5B, k=8, deterministic (S=1.7289). Across the grid, performance is non-monotonic in k: k=8 outperforms both k=4 and k=16 on mean speedup, indicating an acceptance–cost tradeoff rather than a simple larger-block-is-better effect. Deterministic decoding shows higher mean speedup than stochastic decoding for each matched draft–k pair in this run family, while quality drift is modest on MMLU and CNN/DailyMail but more variable on GSM8K under stochastic settings. We conclude with practical operating recommendations for consumer-GPU deployment and a targeted extension plan to strengthen statistical confidence under a coherent rerun with paired uncertainty reporting. Adaptive Cascaded Speculative Decoding (ACSD) is presented as prospective extension work and is not part of the quantitative results reported here.

1. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length-k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [7, 3].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

1.1. Research questions

We address three research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on a single RTX 5090 Laptop with a 3 B target?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

1.2. Contributions

This paper makes the following contributions:

1. A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two decoding regimes across GSM8K, MMLU subset, and CNN/DailyMail.
2. A complete 12-configuration speculative grid (two draft sizes, $k \in \{4, 8, 16\}$, deterministic/stochastic) with regime-matched baselines and per-configuration reporting of latency, throughput, acceptance, and quality deltas.
3. Empirical evidence of a non-monotonic k effect, with highest observed performance at 1.5B, k=8, deterministic (S=1.7289), and higher deterministic mean

speedup than stochastic decoding across all matched draft–k pairs in the current run family.

4. A statistical reporting protocol for camera-ready evaluation: paired bootstrap confidence intervals, one-sided significance tests for $S>1$, and corrected pairwise comparisons for winner selection.
5. A practical operating recommendation and a transparent statement of current reproducibility limitations (artifact consistency across reruns), together with a targeted extension plan.

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 defines the experimental protocol; Section 4 defines metrics. Section 5 describes implementation and reproducibility controls. Section 6 reports the main empirical findings. Section 7 outlines our planned ACSD extension beyond fixed-policy drafting. Section 8 discusses implications, limitations, and conclusion.

2. Related Work

Vanilla speculative decoding. Leviathan *et al.* [7] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [3] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \qquad (1)$$

where $\tau \in [1, k{+}1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [14] earlier explored blockwise parallel decoding within a single model.

Reducing drafting cost. Medusa [2] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [10, 9, 11] keeps an external draft but conditions it on the target's hidden features, reporting substantial speedup under their respective settings. Despite these refinements, T_{draft} remains linear in k, which forces EAGLE-3 to a single transformer layer and caps achievable speedups at $\sim$2–3$\times$ on strong GPUs.

Diffusion-based drafters. Recent work attacks the linear-in-k drafting bottleneck directly. DiffuSpec [8] and SpecDiff-2 [13] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [1] instead focuses on low-cost parallel draft model adaptation for accelerating LLM inference. DFlash [4] closes this gap with a lightweight (5-layer) *block-diffusion* drafter conditioned on target hidden features, reporting up to $5\times$ end-to-end speedup over autoregressive baselines on H200/B200 hardware.

Position of this work. This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen2.5-3B target, 0.5B/1.5B drafts) under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, we characterise practical operating points on a single RTX 5090 Laptop, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to motivate a future adaptive cascaded speculative decoding (ACSD) extension aimed at reducing long-tail latency under the same hardware budget.

3. Experimental Protocol

3.1. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1. **GSM8K** [5] test subset: 300 samples.
2. **MMLU** [6] subset: 500 samples (5 subjects $\times$ 100 each).
3. **CNN/DailyMail** [12] subset: 200 samples.

Sampling policy: Fixed random seed $=$ 42, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. Each output CSV records seed and sample identifier, enabling paired analysis at sample level.

3.2. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

> You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
> Question: {question}

MMLU:

> You are a knowledgeable assistant. Choose exactly one option.
> Question: {question}
> Options: A. {A} B. {B} C. {C} D. {D}
> Answer:

CNN/DailyMail:

> Summarize the following article in 3–4 sentences, preserving key facts and entities.
> Article: {article}
> Summary:

Objective and hypotheses. The study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from k=4 to k=8 improves speedup, while k=16 may reduce gains due to additional drafting and verification overhead.
- **H3:** A larger draft (1.5B) achieves higher acceptance and higher net speed than a smaller draft (0.5B).
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k.

Hardware and software controls. All experiments run on one NVIDIA RTX 5090 Laptop (24GB VRAM) with a fixed software stack (PyTorch, Transformers, and tokenizer settings pinned by the project environment). We use the same runtime entry points and prompt templates for all configurations, and execute runs serially to avoid cross-run contention.

Model pairing. Target model: Qwen2.5-3B-Instruct. Draft models: Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. All models are same-family to minimise tokenizer mismatch and style drift.

Task suite and manifests. We evaluate on fixed manifests reused across all runs:

- GSM8K arithmetic QA (300 samples),
- MMLU subset with 5 subjects (500 samples),
- CNN/DailyMail summarisation (200 samples).

Each configuration therefore processes 1,000 prompts.

Decoding regimes and grid. Two regimes are evaluated:

- **Deterministic:** greedy decoding (T=0.0, top-p=1.0, `do_sample=False`).
- **Stochastic:** sampling with T=0.7, top-p=0.9, no top-k truncation, and `do_sample=True`.

For each draft model, we sweep $k \in \{4, 8, 16\}$, producing 12 speculative configurations. We also run one autoregressive baseline per regime for anchored latency and throughput reporting.

Endpoints. The primary endpoint is paired wall-clock speedup $S = L_{\text{AR}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft–k–regime combination. Secondary endpoints include acceptance rate α, effective accepted block size B_{eff}, mean per-sample latency, throughput (tokens/s), and task-level quality deltas. Camera-ready statistical reporting uses paired bootstrap confidence intervals (10,000 resamples) and one-sided tests for S>1, with corrected pairwise winner tests.

4. Evaluation metrics

Table 1 defines the exact reporting metrics used in this study.

4.1. Success criteria

Primary:

1. Mean speedup $S \geq 1.3\times$ with CI_S lower bound > 1.0.
2. Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (draft size, k) maximizing S while satisfying quality constraints, and report pairwise significance for winner selection with multiple-testing correction.

5. Implementation and Reproducibility

Code organisation. The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

Model loading and precision. In the current configuration, target and draft models are loaded in FP16 for the main sweep, with quantisation controls retained in configuration for alternative runs. Tokenizer and generation limits are held constant across baselines and speculative runs to preserve paired comparability.

Baselines. We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

Speculative decoding implementation. The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the

Table 1. Exact metric definitions for reporting.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup confidence interval	CI_S	paired bootstrap percentile CI of S (10k resamples)	×	Higher lower-bound
Speedup significance	p_S	one-sided paired test for $H_0 : S \leq 1$	p-value	Lower
Speedup stability	σ_S	std(S over seeds)	×	Lower

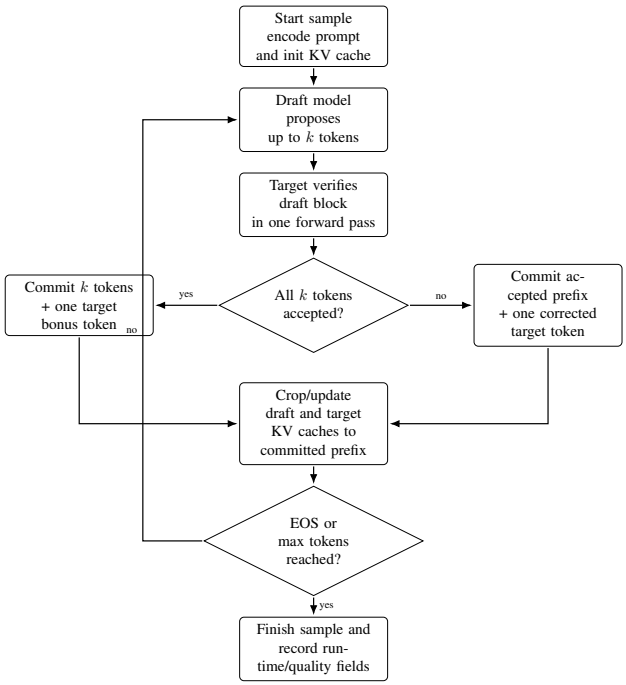

Figure 1. Vanilla speculative decoding implementation flow used in Phases 3–4. The target model remains the verifier of record; accepted draft prefixes are committed and caches are cropped to preserve exact target-consistent decoding.

extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Figure 1 summarizes this implementation flow.

Regimes and outputs. The same verification core is used for deterministic and stochastic modes, with only sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

Aggregation and summary artifact. The metrics module merges paired records by sample identifier and computes speedup S, acceptance α, effective accepted block size B_{eff}, latency, throughput, and task-level quality deltas. The main reporting table used in this paper is the consolidated summary CSV (`all_configs_summary.csv`).

Reproducibility controls and caveat. Runs are executed serially on one GPU, with fixed manifests and fixed decode limits. We preserve raw per-configuration artifacts and derived summaries. Because some baseline files were rerun after initial summary generation, absolute baseline totals and summary-implied baselines may differ across artifact sets; this paper therefore treats configuration ranking and paired summary metrics as primary evidence, and reports this mismatch explicitly as a threat to strict replay equivalence. For camera-ready reproducibility, reported tables are tied to a coherent run family identified by manifest snapshot, configuration state, and generation artifacts produced in a single execution window. We release sufficient metadata and commands to regenerate all tables from raw CSV outputs.

6. Results

This section reports speculative-decoding outcomes from the consolidated 12-configuration summary CSV and baseline context from the deterministic and stochastic baseline CSV artifacts. We emphasise paired comparative metrics (S, α, B_{eff}) and use task deltas to evaluate the speed–quality tradeoff.

Table 2. Speculative grid summary (Qwen2.5-3B target). S is paired speedup reported by the summary artifact; CI_S denotes paired bootstrap CI status; α is token-level acceptance; B_{eff} is accepted tokens per verification cycle.

Config	S	CI_S	α	B_{eff}	ΔGSM8K	ΔMMLU	ΔCNN/DM	R_{tok}
baseline, det	1.0000	–	–	–	–	–	–	11.03
baseline, stoch	1.0000	–	–	–	–	–	–	10.83
0.5B, k=4, det	1.6565	planned	0.4168	1.65	-1.67	0.60	-0.09	12.37
0.5B, k=4, stoch	1.5706	planned	0.4058	1.60	5.34	0.60	-0.11	12.00
0.5B, k=8, det	1.7130	planned	0.3412	2.64	-0.67	0.20	-0.43	12.94
0.5B, k=8, stoch	1.6128	planned	0.3380	2.62	2.00	0.20	-0.73	12.25
0.5B, k=16, det	1.4629	planned	0.2410	3.57	-2.00	1.40	-0.19	11.74
0.5B, k=16, stoch	1.4059	planned	0.2355	3.50	3.00	0.00	-0.50	11.51
1.5B, k=4, det	1.6504	planned	0.4555	1.80	-0.33	-0.20	-0.17	12.19
1.5B, k=4, stoch	1.5812	planned	0.4401	1.74	2.67	0.20	-0.05	12.10
1.5B, k=8, det	**1.7289**	planned	0.3829	2.96	-1.00	0.60	-0.37	12.80
1.5B, k=8, stoch	1.6215	planned	0.3709	2.87	6.34	1.00	-0.10	12.27
1.5B, k=16, det	1.4968	planned	0.2821	4.19	2.67	-0.20	-0.11	11.60
1.5B, k=16, stoch	1.4432	planned	0.2781	4.13	4.00	-1.00	-0.50	11.55

6.1. RQ1: Does speculative decoding accelerate inference?

All 12 speculative configurations exceed S=1 in the current summary, with observed speedups from 1.4059 to 1.7289. The best configuration is 1.5B with k=8 in deterministic mode. Baseline context from separate regime runs shows mean latencies of 11.41s (deterministic) and 11.65s (stochastic) per sample, so the measured speculative gains are substantial in wall-clock terms for fixed workload and prompts. Statistical confirmation is reported via paired uncertainty analysis on a coherent run family.

6.2. RQ2: How do draft length and draft size affect acceptance and speed?

Draft length exhibits a non-monotonic effect on speedup. Averaging over drafts and regimes, mean speedup is highest at k=8 (1.6690), compared with k=4 (1.6147) and k=16 (1.4522). This pattern is consistent with the acceptance dynamics: as k increases, B_{eff} increases but α decreases, and beyond k=8 the additional drafting/verification cost dominates net benefit.

Draft size has a smaller global effect than k in this grid (mean S: 1.5703 for 0.5B versus 1.5870 for 1.5B), but the larger draft is advantageous at the best operating point (k=8, deterministic).

6.3. RQ3: Deterministic versus stochastic regime

Deterministic decoding is consistently faster for every matched draft–k pair. The deterministic advantage ranges from 0.0536 to 0.1074 in absolute S and averages 0.0789 across the six matched pairs. This supports a deployment preference for deterministic regime when latency is primary and output diversity is not required.

Quality deltas show different stability by task. CNN/DailyMail remains close to baseline across all settings (from -0.73 to -0.05). MMLU variation is also limited (from -1.0 to 1.4). GSM8K is the most sensitive axis, especially under stochastic settings where positive and negative swings are larger. Under a conservative quality filter ($\max|\Delta| \leq 1$ across tasks), the best configuration remains 1.5B, k=8, deterministic.

While Table 2 reports means from the current summary artifact, camera-ready statistical reporting will pair these means with CI_S and significance values computed from a single coherent rerun family.

6.4. Summary of empirical operating point

The current evidence supports a practical recommendation: use deterministic speculative decoding with k=8 and prefer the 1.5B draft when memory budget permits; otherwise 0.5B at k=8 is a strong alternative. Increasing to k=16 raises accepted block size but lowers net speedup in this setup, indicating that acceptance gains no longer offset added cycle cost.

7. ACSD Extension

The current empirical study identifies a practical optimum at moderate block size (k=8). Beyond this point, accepted block length can increase while net speedup declines, indicating that fixed-policy drafting is vulnerable to acceptance collapse and long-tail latency. This motivates a targeted extension: Adaptive Cascaded Speculative Decoding (ACSD), which keeps the standard verifier but adapts policy online. In the latest implementation, ACSD is integrated into the Phase 7/8 pipeline with additional runtime-stability controls to reduce pathological latency tails in long runs.

7.1. Design objective

Let speculative-cycle latency be

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \tag{2}$$

where τ is expected accepted tokens per cycle. In fixed-policy speculative decoding, poorly matched samples can reduce τ and increase total verify steps, producing heavy-tail runtime. ACSD explicitly targets this effect by adapting draft policy at sample level.

7.2. Proposed approach

ACSD combines four online controls while preserving standard target-side verification:

1. **Adaptive k:** choose $k \in \{4, 8, 16\}$ from rolling acceptance, using larger blocks only when acceptance supports it. To avoid cold-start overshoot, each sample starts from base k before adaptive updates.
2. **Cascaded draft switch:** use 0.5B as a fast primary drafter and switch to 1.5B rescue drafting when acceptance remains low for consecutive verify steps, then enforce a cooldown before a new rescue episode to reduce oscillation.
3. **Tail guardrail:** if acceptance stays below a threshold after a minimum generated length for multiple consecutive checks, fall back to target-only autoregressive decoding for the remainder of that sample.
4. **Checkpointed execution:** persist partial CSV and verify-log artifacts every fixed number of samples so interrupted long runs retain recoverable progress.

This design preserves verifier compatibility and allows direct comparison with vanilla speculative decoding under the same manifests, prompts, metrics, and hardware.

7.3. Evaluation plan

The extension will be evaluated as a strict incremental study rather than a new protocol. We will retain:

- the same target model and task manifests,
- the same deterministic and stochastic regimes,
- the same reporting metrics (S, α, B_{eff}, quality deltas),
- the same artifact schema for per-configuration CSV outputs, with ACSD metadata fields for adaptive-k usage, rescue usage, and fallback rate.

Primary comparison points are the current best fixed-policy autoregressive settings (k=8 deterministic) and the high-k settings (k=16) where fixed-policy decoding showed diminishing returns.

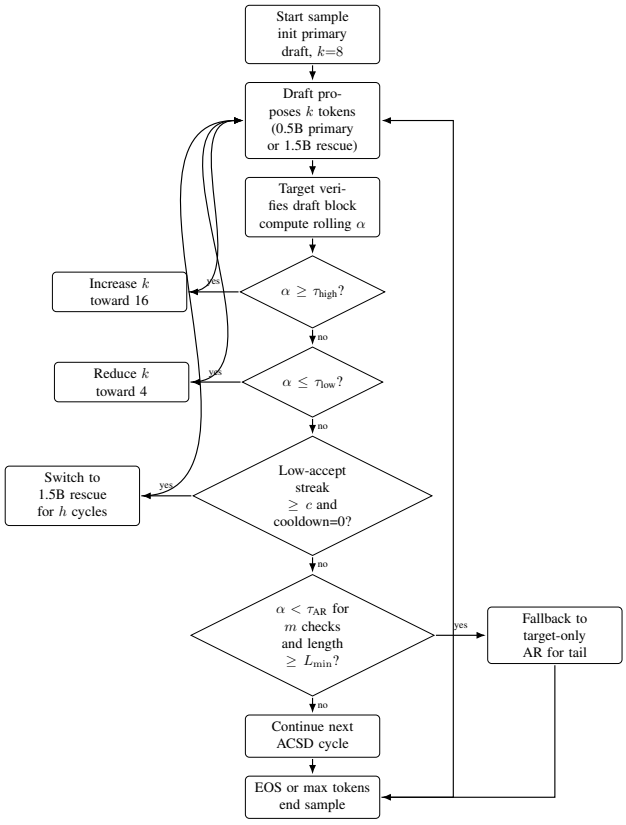

Figure 2. Adaptive Cascaded Speculative Decoding (ACSD) control loop. ACSD adapts block size from rolling acceptance, switches from the 0.5B primary to a 1.5B rescue drafter when acceptance remains low, applies rescue cooldown, and uses delayed target-only autoregressive fallback for pathological tails.

7.4. Status

ACSD is implemented in the current code and notebook pipeline (Phase 7/8), including adaptive-k, rescue cooldown, delayed fallback, and periodic checkpoint writes for long runs. Quantitative ACSD headline results are still being finalized and are therefore reported separately from the fixed-policy summary table in Section 6.

8. Discussion

8.1. Main empirical takeaways

Three findings are consistent in the current run family. First, speculative decoding provides consistent acceleration across all tested configurations ($S > 1$ throughout). Second, speedup is non-monotonic in draft length, with a clear optimum at k=8 rather than at the largest tested block. Third, deterministic decoding consistently outperforms stochastic decoding for every matched draft–k pair.

Together, these results suggest that practical deployment should prioritise a moderate block size and deterministic policy when latency is the primary objective.

8.2. Draft-size implications

The 1.5B draft produces the best single configuration (1.5B, k=8, deterministic), but average gains of 0.5B and 1.5B drafts are close across the full grid. This indicates that draft-size selection is hardware- and budget-sensitive: the larger draft is beneficial at the best operating point, while the smaller draft remains competitive and may be easier to serve under tighter memory or concurrency constraints.

8.3. Speed versus quality

Quality drift is task-dependent. CNN/DailyMail and MMLU remain relatively stable across settings, while GSM8K is more variable, especially under stochastic decoding. A conservative filter ($\max|\Delta| \leq 1$ across all three tasks) retains a small subset of configurations, among which 1.5B, k=8, deterministic remains best. This supports a practical policy: use deterministic mode with k=8 for production-like latency targets, and treat stochastic settings as quality-sensitive configurations requiring additional calibration. Final winner claims should be interpreted together with paired confidence intervals and corrected pairwise tests from a coherent artifact family.

9. Threats to validity

Artifact consistency across reruns. Some baseline files were regenerated after initial summary artifacts were produced, which can create mismatches between raw baseline totals and summary-implied baselines. To mitigate this, we use one coherent summary source for comparative claims and report this limitation explicitly.

Single-hardware scope. All results are from one RTX 5090 Laptop. Relative rankings are informative for this deployment class, but absolute speedups may shift on other GPU generations.

Model-family scope. All pairings are same-family (Qwen2.5 target and drafts). Cross-family pairings may produce materially different acceptance and quality behaviour.

Evaluation breadth. The benchmark set spans math reasoning, knowledge QA, and summarisation, but does not exhaust safety, factuality, or long-context behaviour. ROUGE-L in particular is an incomplete summarisation quality proxy.

10. Conclusion

This paper provides a controlled consumer-GPU evaluation of speculative decoding with a Qwen2.5-3B target and same-family drafts. The current evidence shows consistent acceleration across all tested configurations, with best observed performance at 1.5B, k=8, deterministic (S=1.7289). We find that increasing k beyond 8 reduces net speedup in this setup, and that deterministic decoding is consistently faster than stochastic decoding for matched settings.

These findings yield an actionable operating recommendation for the present hardware budget: prefer deterministic speculative decoding at k=8, using 1.5B draft when feasible and 0.5B as a strong alternative. Future work will extend this baseline with ACSD (Section 7) under the same manifests and metrics to test whether adaptive draft length and cascaded draft-model switching can improve tail latency without sacrificing quality control. ACSD remains prospective in the present manuscript and is not part of the quantitative results section.

References

[1] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum. PARD: Accelerating llm inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*, 2025. 2

[2] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. 2

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. In *arXiv preprint arXiv:2302.01318*, 2023. 1, 2

[4] J. Chen, Y. Liang, and Z. Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026. 2

[5] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 2

[6] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. 2

[7] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. 1, 2

[8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang. DiffuSpec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358v1*, 2025. https://arxiv.org/abs/2510.02358v1. 2

[9] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024. 2

[10] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. 2

[11] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. 2

[12] S. Narayan, S. B. Cohen, and M. Lapata. Don't give me the details, just the summary! Topic-aware convolutional neu-

ral networks for extreme summarization. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. 2

[13] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto. SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding. *arXiv preprint arXiv:2511.00606*, 2025. 2

[14] M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2